

Computer Networks

Assignment 02

Field	Details
Name	Aryan Yadav
UID	24BCF10079
Subject	Computer Networks
Assignment No.	02

Q1. IP Address, Classes, Subnetting and Supernetting

What is an IP Address?

An IP (Internet Protocol) Address is a unique numerical label assigned to every device connected to a network that uses the Internet Protocol for communication. It identifies the device (host) and the network it belongs to, so that data packets can be correctly routed from source to destination. An IPv4 address is 32 bits long, usually written as four decimal numbers separated by dots (e.g., 192.168.1.10), where each number represents 8 bits (an octet).

Why do we need classes in IPv4 addressing?

When IPv4 was designed, the 32-bit address space had to be divided among networks of very different sizes - from huge organisations needing millions of hosts to small offices needing only a few. Classful addressing splits the address space into fixed classes (A-E) so that the network and host portions can be identified easily just by looking at the first few bits of the address. This made address allocation and routing simple, although it later caused wastage of addresses, which is why CIDR (subnetting/supernetting) was introduced.

32-bit IP Address

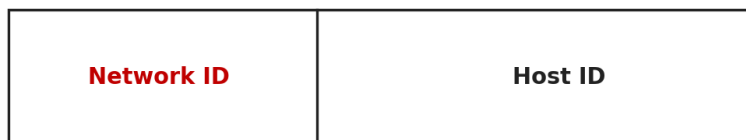


Fig 1.1 - Every IPv4 address is divided into a Network ID part and a Host ID part

The Five Classes of IPv4 Address

Class	First Bits	First Octet Range	Default Mask	No. of Networks	Hosts / Network
A	0	1 - 126	255.0.0.0	128 (2^7)	16,777,214 ($2^{24} - 2$)

Class	First Bits	First Octet Range	Default Mask	No. of Networks	Hosts / Network
B	10	128 - 191	255.255.0.0	16,384 (2^{14})	65,534 ($2^{16} - 2$)
C	110	192 - 223	255.255.255.0	2,097,152 (2^{21})	254 ($2^8 - 2$)
D	1110	224 - 239	Not applicable (multicast)	-	-
E	1111	240 - 255	Not applicable (reserved)	-	-

Note: Class D is reserved for multicast groups and Class E is reserved for experimental / future use, so they are not assigned to hosts or networks. Two host addresses are subtracted in Class A, B and C because the all-0s address identifies the network itself and the all-1s address is the broadcast address.

Subnetting

Subnetting is the process of dividing one large network into several smaller networks called subnets, by borrowing bits from the host portion of the address to extend the network portion. This helps in efficient use of addresses, reduces broadcast traffic and makes network management easier.

Example: Consider the network 192.168.1.0/24 (256 addresses). If we borrow 2 host bits to make it a /26 network, it gets divided into 4 subnets of 64 addresses each:

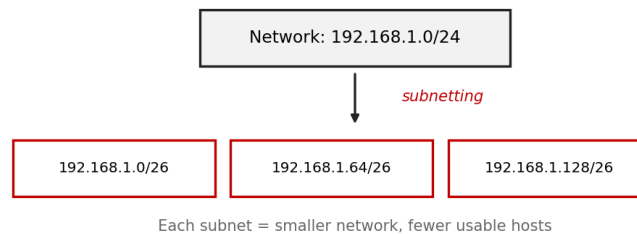


Fig 1.2 - One /24 network subnetted into four /26 subnets

Supernetting

Supernetting is the reverse process - combining several smaller (contiguous) networks into a single, larger network by reducing the number of bits in the subnet mask. It is mainly used in CIDR (Classless Inter-Domain Routing) to reduce the size of routing tables at the internet backbone level.

Example: Four Class C networks 192.168.0.0/24, 192.168.1.0/24, 192.168.2.0/24 and 192.168.3.0/24 can be combined into a single supernet 192.168.0.0/22:

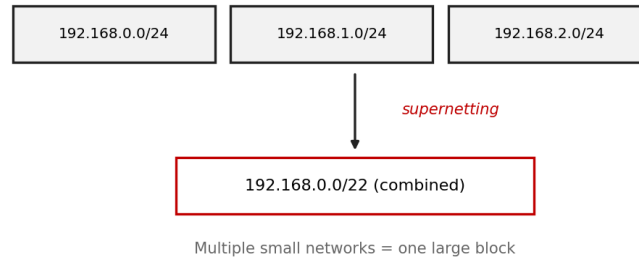


Fig 1.3 - Four /24 networks combined into one /22 supernet

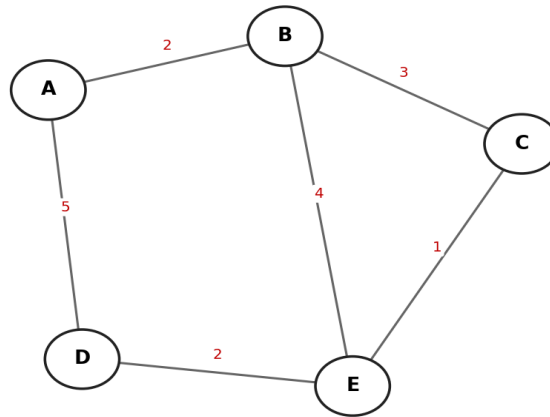
Q2. Link State vs Distance Vector Routing

Routing algorithms decide the best (least cost) path a packet should take from source to destination. The two main types used in dynamic routing are Distance Vector and Link State routing.

Difference between Link State and Distance Vector Routing

Basis	Distance Vector Routing	Link State Routing
Information shared	Its entire routing table (distance to all known networks)	Only the state/cost of its own directly connected links
Shared with	Only directly connected neighbours	Flooded to all routers in the area
Algorithm used	Bellman-Ford Algorithm	Dijkstra's Algorithm
Network view	Knows the network indirectly, through neighbours	Builds a complete, independent map of the topology
Convergence speed	Slow, prone to routing loops / count-to-infinity	Faster and more accurate
Example protocol	RIP (Routing Information Protocol)	OSPF (Open Shortest Path First)

How each algorithm finds the least cost path



Sample network graph (numbers = link cost)

Fig 2.1 - Sample network used for both examples

Distance Vector (Bellman-Ford) example

Each router starts by knowing only the cost to its direct neighbours and sets the cost to all other nodes as infinity. It then repeatedly exchanges its distance vector with neighbours and updates its table if a neighbour offers a cheaper path, using the formula:

$$\text{Distance}(A \text{ to } E) = \min [\text{Cost}(A, \text{neighbour}) + \text{Distance}(\text{neighbour to } E)]$$

For example, router A initially only knows B (cost 2) and D (cost 5). After exchanging vectors, A learns from B that B can reach E via C at a cost of $3+1=4$, so A updates its route to E as A-B-C-E with cost $2+4=6$. This process repeats every cycle until no further improvement is possible (convergence).

Link State (Dijkstra) example

Every router floods a Link State Advertisement (LSA) describing its direct links and their costs to all other routers, so every router ends up with the same complete map of the network (as shown in Fig 2.1). Each router then independently runs Dijkstra's algorithm on this map to build a shortest path tree rooted at itself.

For example, to find the shortest path from A to E, Dijkstra's algorithm examines all possible paths: A-D-E ($5+2=7$), A-B-E ($2+4=6$) and A-B-C-E ($2+3+1=6$). The algorithm picks the lowest cost path(s), here 6, giving A-B-E or A-B-C-E as the least cost routes - and every router reaches this same conclusion independently because they all have identical topology information.

Q3. Congestion Control and Fragmentation

What is Congestion in a Network?

Congestion occurs when too many packets are present in a part of a network (a router or a link) at the same time, exceeding its handling capacity. This leads to increased delay, packet loss, and a drop in overall network

throughput - in severe cases, throughput can even collapse close to zero even though the offered load keeps increasing.

Why do we need to control congestion?

- To prevent packet loss and unnecessary retransmissions
- To keep end-to-end delay and jitter within acceptable limits
- To make fair use of bandwidth among all users/flows
- To avoid throughput collapse (congestion collapse) of the network
- To use buffers and links efficiently instead of wasting resources on dropped packets

Leaky Bucket vs Token Bucket

Both are traffic shaping algorithms that control the rate at which packets are sent into the network, but they work differently:

Basis	Leaky Bucket	Token Bucket
Output rate	Always constant, irrespective of input burst	Can vary; allows sending at a higher rate if tokens are available
Handling of bursty traffic	Smooths out bursts completely - no burst allowed	Allows controlled bursts up to the number of saved-up tokens
Working principle	Packets are queued in a bucket and leaked out at a fixed rate	Tokens are generated at a fixed rate; a packet is sent only if a token is available
Flexibility	Rigid, less efficient for bursty applications	More flexible and efficient, widely used in practice

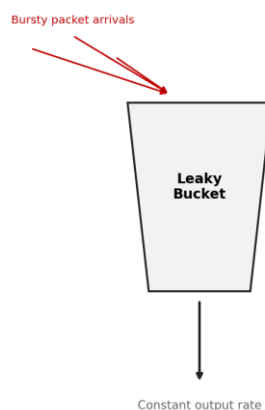


Fig 3.1 - Leaky Bucket

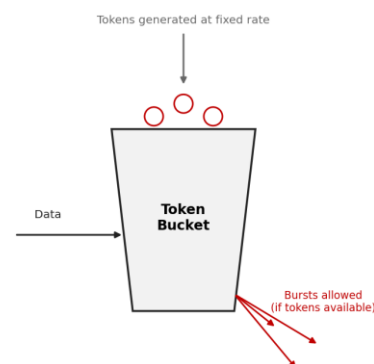
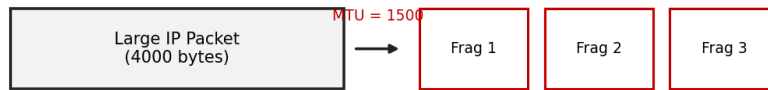


Fig 3.2 - Token Bucket

Why do we need Fragmentation?

Different networks and data-link technologies support different maximum packet sizes, known as the MTU (Maximum Transmission Unit) - for example, Ethernet typically supports an MTU of 1500 bytes. When an IP

packet is larger than the MTU of a link it must cross, it cannot be sent as-is. Fragmentation is the process of breaking such a large packet into smaller fragments, each small enough to fit within the MTU of the outgoing link. These fragments travel independently and are reassembled into the original packet at the destination host.



Packet is split into smaller fragments to fit the MTU of a link

Fig 3.3 - A large packet fragmented to fit a smaller MTU

Without fragmentation, a packet larger than a link's MTU would simply be dropped, so fragmentation is essential to ensure reliable delivery of data across networks made up of links with different MTU values.